SLIIT UNI

THE KNOWLEDGE UNIVERSITY

09 – Network Programming

IE2004 Computer Networks

2026 July

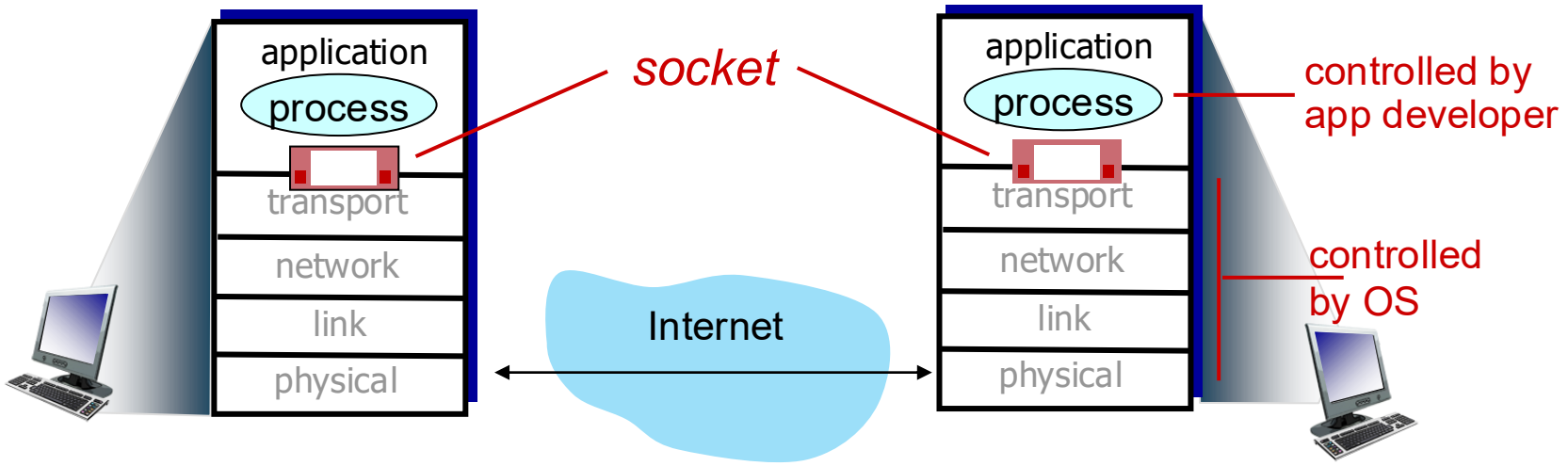
Lesson outline

- Socket programming
- Socket programming with UDP
- UDP client
- UDP server
- Socket programming with TCP
- TCP client
- TCP server
- Socket timeouts
- API: socket object and its methods

Socket programming

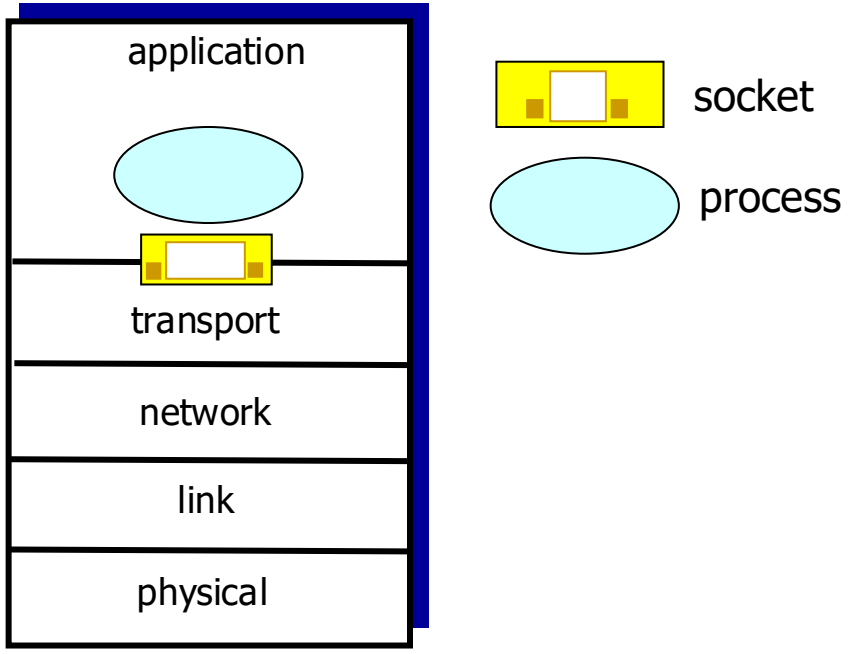
goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Sockets

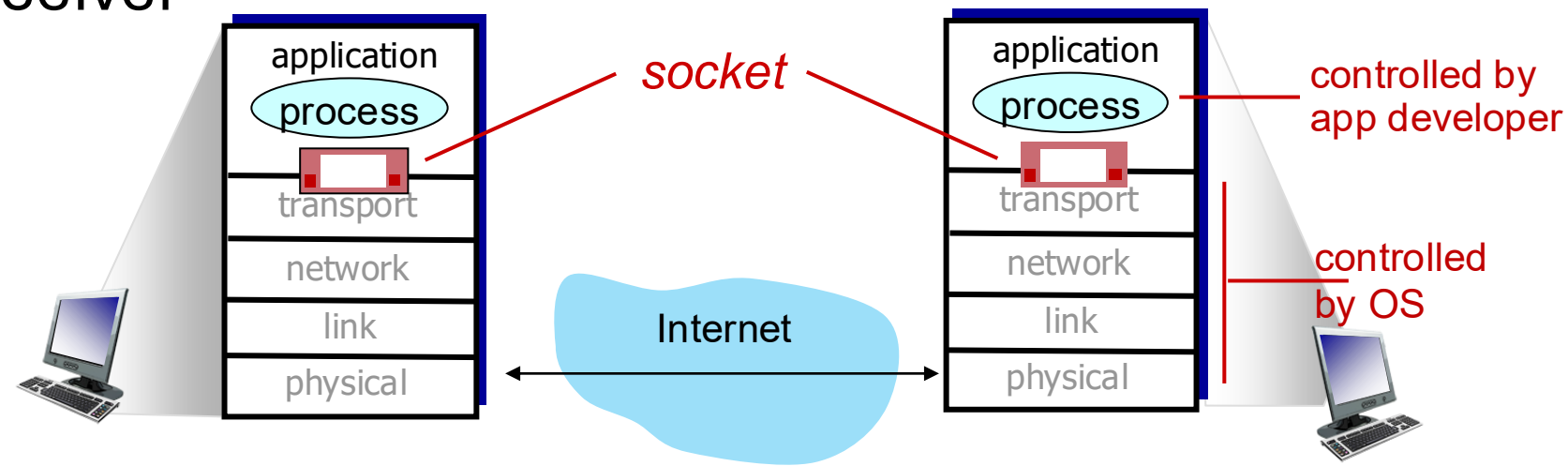
- A process in the Application Layer connects to the Transport Layer via a socket
  - Process = running app / part of a running app
- A socket is an endpoint for sending or receiving data across a network
- Socket created by application layer process
  - socket is connected to that process
- A socket is identified by the protocol, IP address and port number
  - Plus remote IP and port number in TCP
  - The Port number is specified using 16-bits
  - Port number can range from 0 to 65535



Recall from topic 7 (Transport layer)

Sockets (ctd)

- A socket is like a door
  - process sends/receives messages to/from its socket
  - sending process pushes message out through the door
  - sending process relies on transport layer on other side of door to deliver message to socket at receiving process
  - communication involves two sockets: one at sender and one at receiver



Socket programming

Two socket types for two transport services:

- *UDP:* unreliable datagram
- *TCP:* reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly includes IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides unreliable transfer of groups of bytes (“datagrams”) between client and server processes

Client/server socket interaction: UDP



server (running on serverIP)



client

create socket, port= x:

```
serverSocket =
   socket(AF_INET,SOCK_DGRAM)
```

read message from serverSocket

write reply to serverSocket specifying client address, port number

create socket:

```
clientSocket =
   socket(AF_INET,SOCK_DGRAM)
```

Create message; send message via clientSocket specifying serverIP address and port;

read message from clientSocket

close clientSocket

Example app: UDP client

Python UDPClient

include Python's socket library →
```
from socket import *
```
```
serverName = 'hostname'
serverPort = 12000
```
create UDP socket →
```
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
```
get user keyboard input →
```
message = input('Input lowercase sentence:')
```
send into socket the message, server name/IP, port →
```
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
```
read reply data (bytes) from socket →
```
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
```
print out received string and close socket →
```
print(modifiedMessage.decode())
clientSocket.close()
```

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket →

bind socket to local port number 12000 →

loop forever →

Read from UDP socket into message, getting client's address (client IP and port) →

send upper case string back to this client →

Socket programming with TCP

Client must contact server

- server process must be running
- server must have created socket that welcomes client's contact

Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when a client connects, server TCP creates new socket for server process to communicate with that particular client
  - allows server to talk with multiple clients
  - client source port # and IP address used to distinguish clients (4-Tuple)

Application viewpoint

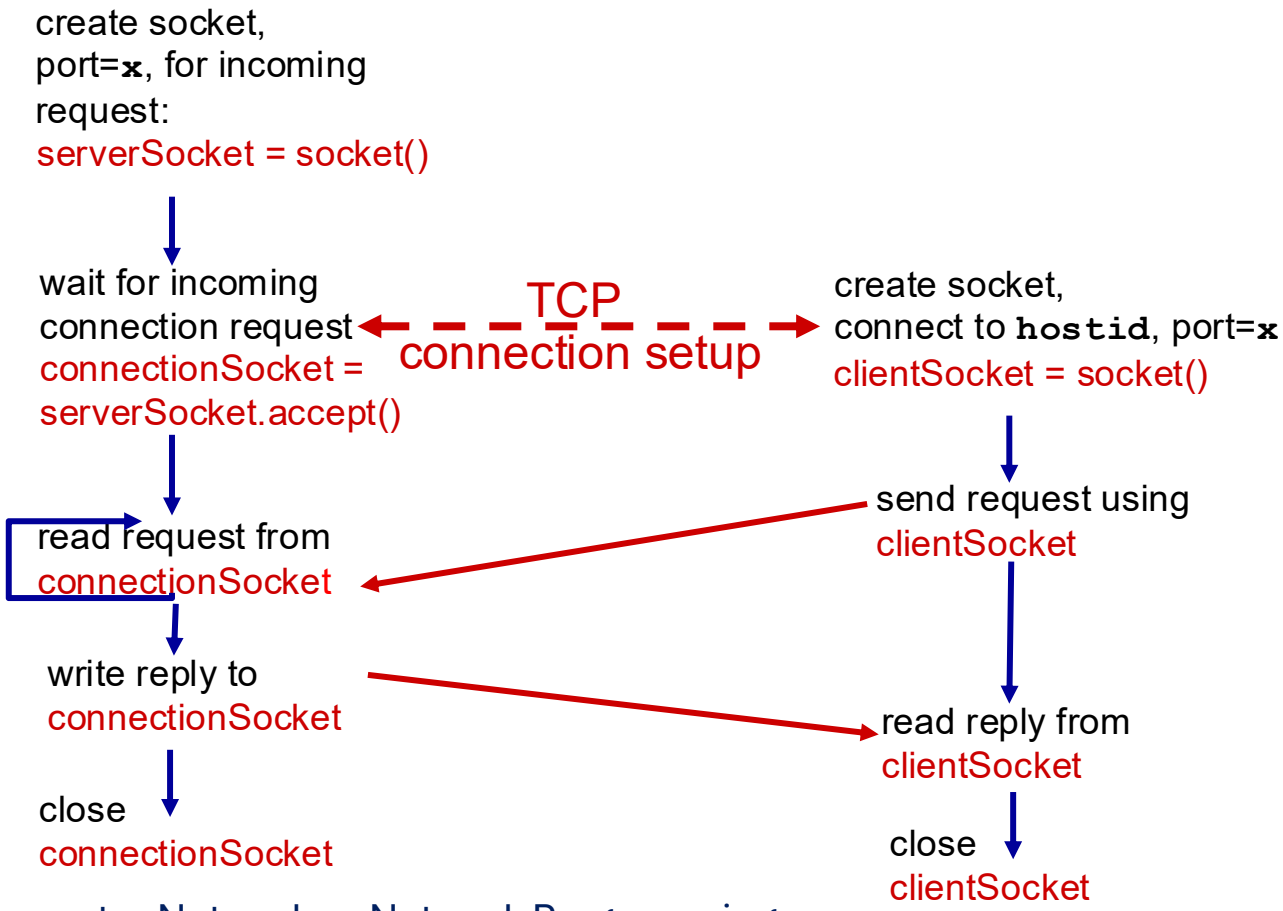
TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/server socket interaction: TCP



server (running on hostid)

client 



Example app: TCP client

Python TCPClient

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

create TCP socket →

connect socket to server port 12000 →

Send data into socket →

Read received data from socket →

Example app: TCP server

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                          encode())

    connectionSocket.close()
```

create TCP welcoming socket →

server begins listening for incoming TCP requests →

loop forever →

server waits on accept() for incoming requests, new socket created on return →

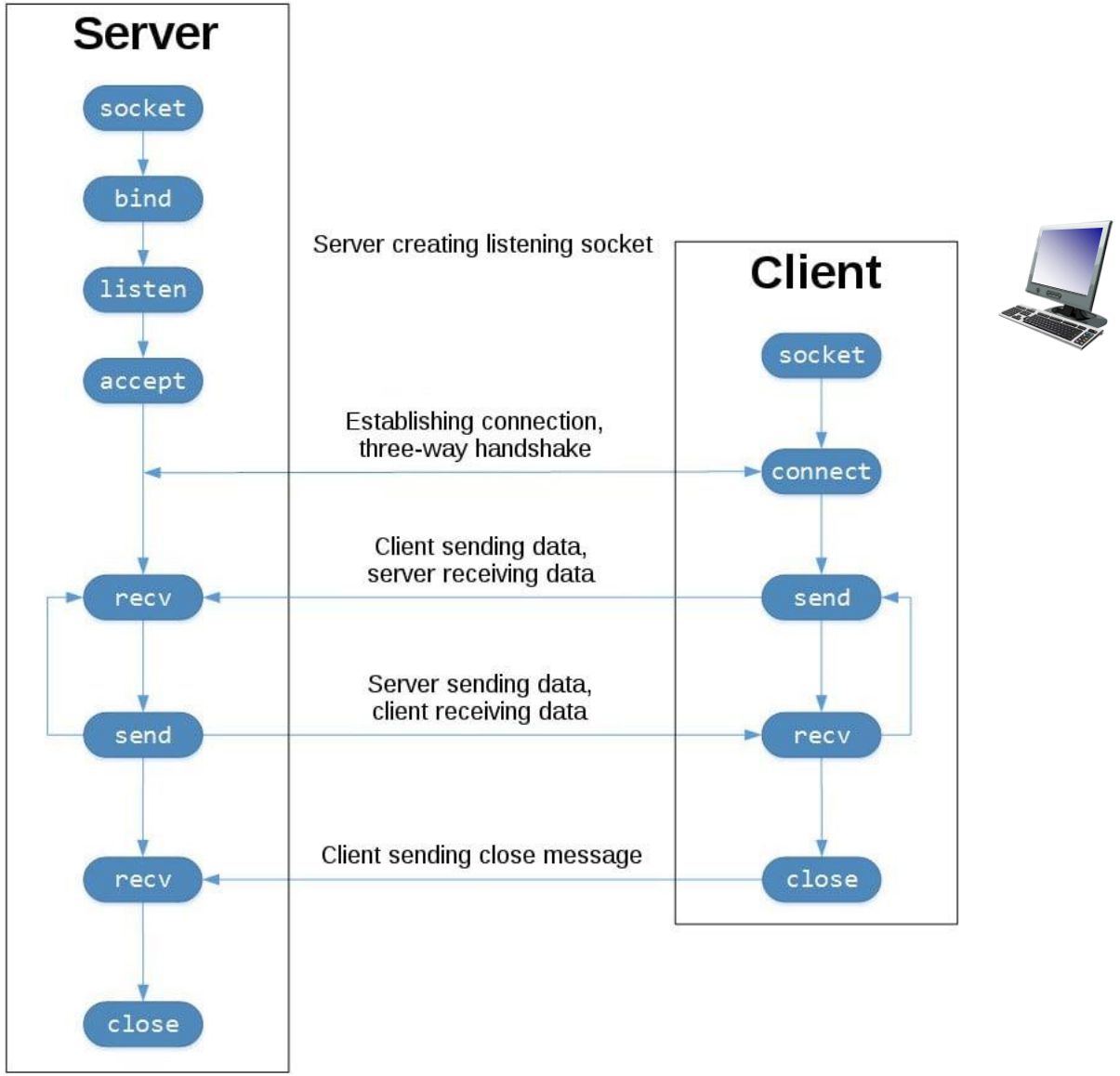
read bytes from socket (but not address as in UDP) →

send modified message into socket →

close connection to this client (but *not* welcoming socket) →

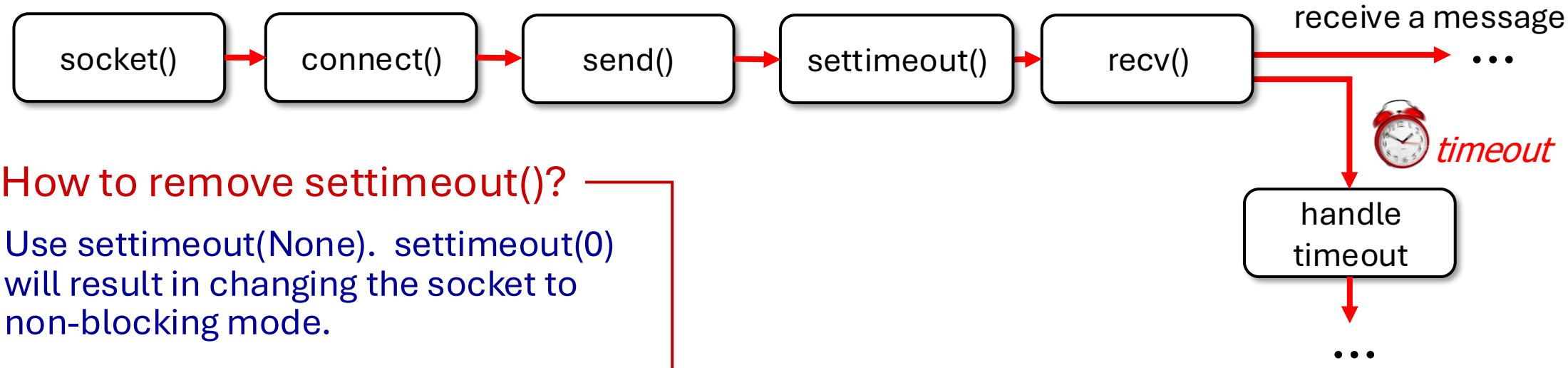
Client/server socket interaction: TCP





Socket programming: waiting for multiple events

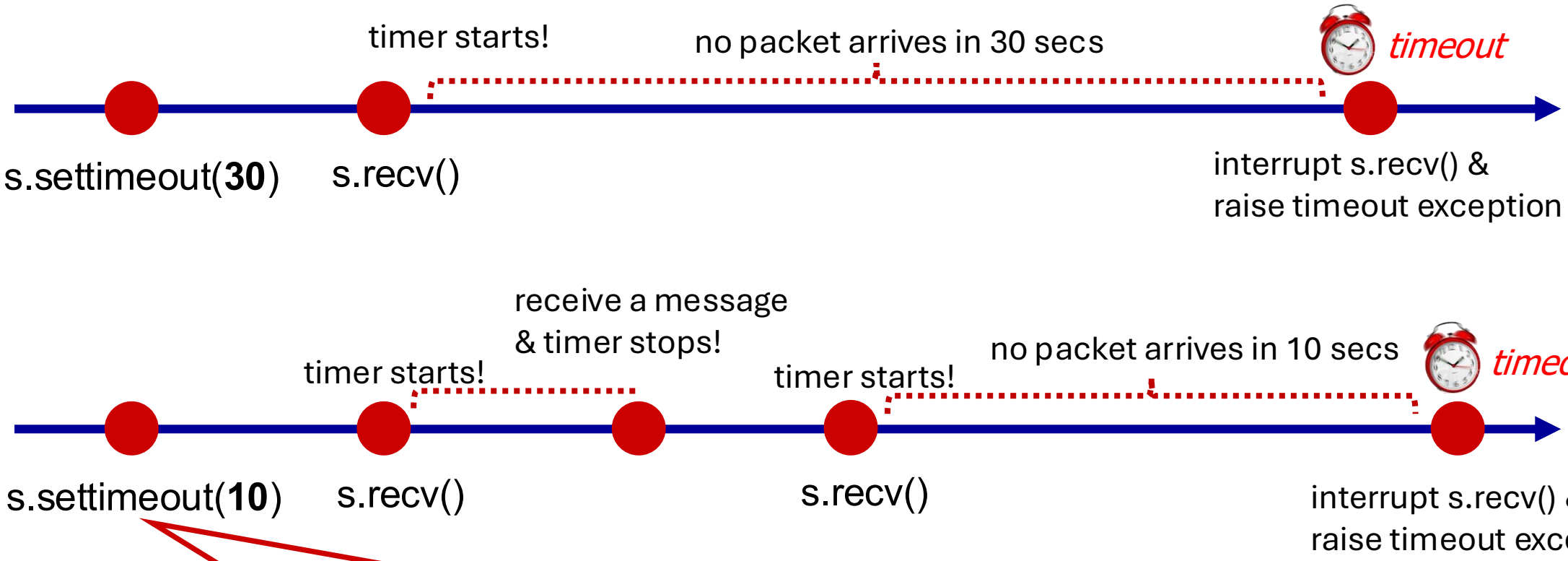
- Often a program must wait for one of several events to happen, e.g.,:
  - wait for either (i) a reply from remote end of the socket, or (ii) timeout: timer
  - wait for replies from several different open sockets: select(), multithreading
- timeouts are used extensively in networking
- using timeouts with Python socket:



How to remove settimeout()?

Use settimeout(None). settimeout(0) will result in changing the socket to non-blocking mode.

How Python socket.settimeout() works?



Python try-except block

Execute a block of code, and handle “exceptions” that may occur when executing that block of code

```
try:
    <do something>
except <exception>:
    <handle the exception>
```

Executing this try code block may cause exception(s) to catch. If an exception is raised, execution jumps from jumps directly into except code block

this except code block is only executed if an <exception> occurred in the try code block (note: except block is required with a try block)

Socket programming: socket timeouts

Toy Example:



- A shepherd boy tends his master's sheep.
- If he sees a wolf, he can send a message to villagers for help using a TCP socket.
- The boy found it fun to connect to the server without sending any messages. But the villagers don't think so.
- And they decided that if the boy connects to the server and doesn't send the wolf location within 10 seconds for three times, they will stop listening to him forever and ever.

Python TCPServer (Villagers)

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
counter = 0
while counter < 3:
    connectionSocket, addr = serverSocket.accept()
    connectionSocket.settimeout(10)
    try:
        wolf_location = connectionSocket.recv(1024).decode()
        send_hunter(wolf_location) # a villager function
        connectionSocket.send('hunter sent')
    except timeout:
        counter += 1
connectionSocket.close()
```

set a 10-seconds timeout on all future socket operations →

timer starts when recv() is called and will raise timeout exception if there is no message within 10 seconds. →

catch socket timeout exception →

API: socket object and methods

- `socket()`
- `.bind()`
- `.listen()`
- `.accept()`
- `.connect()`
- `.connect_ex()`
- `.send()`
- `.recv()`
- `.close()`
- See https://docs.python.org/3/library/socket.html for more information

Socket API functions

- `socket(family=AF_INET, type=SOCK_STREAM, proto=0, fileno=None)`
- Create a new socket using the given address family, socket type and protocol number
- The address family can be AF_INET (the default), AF_INET6, AF_UNIX, …
- The socket type should be SOCK_STREAM (the default), SOCK_DGRAM, SOCK_RAW or perhaps one of the other SOCK_ constants
- The protocol number is usually zero
- fileno is used for creating file sockets (not network sockets)
- Returns the newly created socket object (of class socket)

socket object methods

- `socket.bind(address)`
  - Bind the socket to address
  - The socket must not already be bound
  - The format of address depends on the address family
- `socket.listen([backlog])`
  - Enables a server to accept connections
  - If backlog is specified, it specifies the number of unaccepted connections that the system will allow before refusing new connections
  - If not specified, a default reasonable value is chosen
  - Backlog must be at least 0 (if it is lower, it is set to 0)

socket object methods (3)

- `socket.accept()`
  - Accept a connection
  - The socket must be bound to an address and listening for connections
  - The return value is a pair (conn, address)
  - conn is a new socket object usable to send and receive data on the connection
  - Address is the address bound to the socket on the other end of the connection
- `socket.connect(address)`
  - Connect to a remote socket at the provided address
  - The format of address depends on the address family
  - Raises exceptions if connection is not successful or if a timeout occurs

socket object methods (4)

- `socket.connect_ex(address)`
  - Like connect(address), but return an error indicator instead of raising an exception for errors
  - The error indicator is 0 if the operation succeeded, otherwise the value of the errno variable
  - The format of address depends on the address family
- `socket.recv(bufsize,[flags])`
  - Receive data from the socket
  - The return value is a bytes object representing the data received
  - The maximum amount of data to be received at once is specified by bufsize
  - A returned empty bytes object indicates that the client has disconnected
  - flags is an optional argument; it defaults to zero.

socket object methods (5)

- `socket.send(bytes,[flags])`
  - Send data to the socket
  - The socket must be connected to a remote socket
  - Returns the number of bytes sent
  - Applications are responsible for checking that all data has been sent
  - If only some of the data was transmitted, the application needs to attempt delivery of the remaining data
  - The optional flags argument has the same meaning as for recv() above
- `socket.close()`
  - Mark the socket closed
  - The remote end will receive no more data (after queued data is flushed)
  - Sockets are automatically closed when they are garbage-collected, but it is recommended to close() them explicitly, or to use a with statement around them

address parameter

- For AF_INET address family (IPv4), a pair (host, port) is used
- host can be one of
  - a hostname like ‘www.sliit.lk’ => (host,port) = (‘www.sliit.lk’,80)
  - an IPv4 address like '100.50.200.5’ => (host,port) = (‘100.50.200.5’,25)
- For IPv4 addresses, two special forms are accepted instead of a host address:
  - '' represents INADDR_ANY, which is used to bind to all interfaces
  - the string '<broadcast>' represents INADDR_BROADCAST
- For AF_INET6 address family, a four-tuple (host, port, flowinfo, scope_id) is used

References

- Chapter 2 - The Application Layer
  James F. Kurose and Keith W. Ross, Computer Networking – A Top-Down Approach, (8th Edition), Pearson, 2020
- Python socket programming documentation, https://docs.python.org/3/library/socket.html

Lecture slides prepared by

Nimal Ratnayake
Dept of Computer Systems Engineering
SLIIT

First version: 2024 July
Revision 1: 2025 February
Revision 2: 2025 July